

Structs, Methods, Interfaces & Errors Cheatsheet

Lessons: [09](#) — [Structs](#) · [10](#) — [Pointers & Methods](#) · [11](#) — [Interfaces](#) · [12](#) — [Errors](#)

Examples: [09](#) · [10](#) · [11](#) · [12](#)

Covers: struct literals & tags, pointers, method sets, embedding, interface satisfaction, the `errors` package

Legend: `[*]` = real Go API that the lessons have not covered yet

STRUCTS

```
type User struct { ID int; Name string }    the declaration
u := User{ID: 1, Name: "Ann"}              keyed literal – always use this form
u := User{1, "Ann"}                        positional: breaks when fields are added
var u User                                 every field at its zero value
u.Name                                     field access (works through pointers too)
p := &User{Name: "Ann"}                   pointer to a fresh struct – the constructor shape
new(User)                                  *User with zero fields (rare; &User{} reads better)
struct{}{}                                 the empty struct: zero bytes, used as a signal
u1 == u2                                   comparable IF every field is comparable
anon := struct{ A int }{1} [*]            an anonymous struct – handy in tests
(a struct is a VALUE: assigning or passing it copies every field)
```

STRUCT TAGS

```
Name string `json:"name"`                 rename the field in JSON
Email string `json:"email,omitempty"`     drop it when empty
Secret string `json:"-"`                  never marshal this field
ID int `json:"id" db:"id"`                several tags, space-separated
Age int `json:",string" [*]`              encode the number as a JSON string
`validate:"required,email" [*]           third-party validators read tags the same way
(tags are just strings read by reflection – a typo fails silently)
```

EMBEDDING (composition, not inheritance)

```
type Admin struct { User; Level int }      embed by TYPE, no field name
a.Name                                     promoted from the embedded User
a.User.Name                               the explicit path (always available)
a := Admin{User: User{...}}               the literal must name the embedded type
type S struct { *sync.Mutex } [*]         embedding a pointer works too
type Repo struct { DB }                   embedded interface: satisfy it by delegation
outer method wins                         an Admin.String() shadows User.String()
(embedding promotes METHODS too – that's how interfaces get satisfied)
```

POINTERS

```
&x                                         the address of x -> *T
*p                                         the value at p (dereference)
p.Name                                    auto-dereference: (*p).Name is never needed
var p *User                               nil pointer; dereferencing it panics
new(T)                                    allocate a zero T, return *T
p == nil                                  always check before dereferencing
func f(u *User)                           the callee can MUTATE the caller's struct
func f(u User)                             the callee gets a copy
&slice[0] [*]                             a pointer into a slice's backing array
(no pointer arithmetic; escape analysis decides stack vs heap, not you)
```

METHODS & RECEIVERS

<code>func (c Counter) Value() int</code>	VALUE receiver: operates on a copy
<code>func (c *Counter) Inc()</code>	POINTER receiver: mutates the original
<code>c.Inc()</code>	Go auto-takes <code>&c</code> when <code>c</code> is addressable
<code>p.Value()</code>	Go auto-dereferences <code>p</code>
methods on named types only	not on <code>int</code> , and not on other packages' types
one receiver name per type	pick a short one and use it everywhere
<code>func (c *Counter) String() string</code>	satisfying <code>fmt.Stringer</code>

THE METHOD SET RULE (memorize)

value <code>T</code> holds	methods with VALUE receivers
pointer <code>*T</code> holds	methods with value AND pointer receivers
so: <code>*T</code> satisfies more interfaces than <code>T</code> does	
<code>var s Shape = Circle{}</code>	OK if <code>Area()</code> has a value receiver
<code>var s Shape = Circle{}</code>	COMPILE ERROR if <code>Area()</code> has a pointer receiver
<code>var s Shape = &Circle{}</code>	that one works
map/slice elements	not addressable: <code>m["k"].Inc()</code> won't compile
rule of thumb	if ANY method needs a pointer, give them ALL pointers

METHOD VALUES & EXPRESSIONS

<code>f := u.Name</code>	a METHOD VALUE: the receiver is bound NOW, and copied if the receiver is a value type
<code>f()</code>	calls it later, on that captured receiver
<code>g := (*User).SetName</code>	a METHOD EXPRESSION: an ordinary func whose FIRST parameter is the receiver
<code>g(&u, "Ann")</code>	so it can be passed where a <code>func(*User, string)</code> is wanted
<code>http.HandlerFunc(h.ServeIt)</code>	the everyday use: a bound method as a callback
(a method value on a value receiver snapshots the receiver – later mutations are invisible to it; use a pointer receiver if that matters)	

POINTER TRAPS WORTH MEMORIZING

append can move the array	<code>p := &s[0]; s = append(s, x) -></code> <code>p</code> may now point at the OLD array. Never hold a pointer across an append.
range gives you a COPY	for <code>_, v := range s { v.N++ }</code> changes nothing; use <code>s[i].N++</code> , or range over the index
map values aren't addressable	<code>m[k].Field = v</code> won't compile use <code>map[K]*V</code> , or read-modify-write the whole value
<code>&loopVar</code>	safe since Go 1.22 (per-iteration variables); before that, every pointer aliased the same variable
pointer as a map key	<code>map[*T]bool</code> keys on IDENTITY, not contents – exactly what you want for a set of live objects
<code>*T</code> vs <code>T</code> in a slice	<code>[]*T</code> lets you mutate in place; <code>[]T</code> is denser and cheaper for the GC (see the low-latency sheet)
<code>weak.Pointer[T]</code>	<code>[*]</code> Go 1.24+: reference a value without keeping it alive – for caches that must not leak
<code>unsafe.Pointer</code>	<code>[*]</code> the escape hatch: no type safety, no GC guarantees

INTERFACES

```
type Shape interface { Area() float64 }    a set of method signatures
implicit satisfaction                      no `implements` keyword – matching methods is enough
var s Shape = Circle{}                    assignment is the check
any                                       alias for interface{} – satisfied by everything
var _ Shape = (*Circle)(nil) the compile-time assertion idiom
accept interfaces, return structs        the API design rule
keep interfaces small                    1–3 methods; define them where they're USED
type Reader interface { Read(p []byte) (int, error) } the stdlib's shape
optional interface                      feature detection: take the basic interface, then ask
                                       for more at runtime
    if f, ok := w.(http.Flusher); ok { f.Flush() }
    if rf, ok := dst.(io.ReaderFrom); ok { return rf.ReadFrom(src) }
                                       – how io.Copy uses a fast path when one exists,
                                       and how a decorator can pass capabilities through
```

TYPE ASSERTIONS & SWITCHES

```
v := x.(Circle)                          assert; PANICS if x isn't a Circle
v, ok := x.(Circle)                       the comma-ok form: never panics
v, ok := x.(Shape)                         assert to another interface
switch v := x.(type) {                    type switch: v has the branch's type
case Circle: ...                           one concrete type
case Shape: ...                           an interface type also matches
case nil: ...                             x holds no value at all
default: ...                             everything else
}
```

THE NIL INTERFACE TRAP

```
an interface is 2 words                  (type, value)
iface == nil                             only when BOTH are nil
var p *MyErr = nil                       a typed nil pointer
var e error = p                           e != nil – type is set, value is nil
return p                                  the classic bug: a nil *MyErr becomes a non-nil error
fix                                       return nil explicitly, or keep the concrete type out
```

THE STDLIB INTERFACES WORTH KNOWING

```
error                                    Error() string
fmt.Stringer                            String() string – used by %v and Println
io.Reader / io.Writer                   Read(p) (n, err) / Write(p) (n, err)
io.Closer / io.ReadWriter               Close() error / both of the above
sort.Interface                          [*] Len, Less, Swap
json.Marshaler                          [*] MarshalJSON() ([]byte, error)
http.Handler                            ServeHTTP(w, r)
driver.Valuer / sql.Scanner             [*] custom DB column types
```

ERRORS: creating

```
errors.New("not found")      a simple sentinel value
fmt.Errorf("open %s: %w", name, err)  WRAP: keeps the cause in the chain
fmt.Errorf("bad input: %v", err)      formats but BREAKS the chain
var ErrNotFound = errors.New("not found")  package-level sentinel
type NotFoundError struct { ID int }      a custom error type ...
func (e *NotFoundError) Error() string    ... needs this one method
errors.Join(err1, err2)  [*] Go 1.20+: several errors as one
func (e *E) Unwrap() error [*] make a custom type wrappable
```

ERRORS: inspecting

```
if err != nil { return err } handle or return – never both, never neither
errors.Is(err, ErrNotFound) is this sentinel anywhere in the chain?
var e *NotFoundError; errors.As(err, &e)  pull a typed error out of the chain
errors.Unwrap(err)      [*] one step down the chain
err == ErrNotFound      WRONG once anything wraps it – use errors.Is
_ = f()                  explicitly ignoring an error (say why in a comment)
errors.Is(err, context.Canceled)  the shape for cancellation checks
```

ERROR STYLE

```
lowercase, no punctuation  "open config: permission denied"
add context going up       each layer prepends what IT was doing
wrap with %w at boundaries so callers can still use Is/As
don't log AND return       the caller will log it once
sentinel for expected cases ErrNotFound, ErrAlreadyExists
custom type when you need data  status codes, field names, IDs
panic only for programmer bugs  not for a bad request body
(the error message is read by a human at 3am – make it say what failed)
```

TRAPS & MEMORIZE

```
positional struct literals  break silently when a field is inserted
copying a struct with a mutex  go vet copylocks; embed *sync.Mutex or don't copy
value receiver mutation      changes the copy and is silently lost
mixing receiver kinds        confusing method sets; pick one per type
map[K]Struct field write     m["k"].F = v doesn't compile – not addressable
interface holding a typed nil != nil; the #1 error-handling bug
big interfaces               harder to fake in tests; keep them at 1–3 methods
defining interfaces early    define them in the CONSUMER, when you need them
%v instead of %w             loses errors.Is/As for every caller above you
comparing wrapped errors     use errors.Is, not ==
struct tag typos             fail silently – there is no compiler check
```